FindIt System Manual

Version 0.3.12 (Manual)

Build Date: December 5, 2025

FindIt: A LoRaWAN-Based Asset Tracking System

by

Ryan Davis, Cory Vitanza, and Roy Tas

Group 2

Stevens Institute of Technology

Fall 2025

Version 0.3.12 (Manual)

Build Date: December 5, 2025

Group 2
Stevens Institute of Technology

FindIt: A LoRaWAN-Based Asset Tracking System

Ryan Davis, Cory Vitanza, and Roy Tas

Group 2
Stevens Institute of Technology

Table 1: Document Update History

Date	**Updates**
12/05/2025	RD, RT, CV: • Added Weekly Report 12 to appendix. • Updated Development Notes for Development board configurations
11/18/2025	RD, RT, CV: • UML Class diagrams for our Logical view, a package diagram for our development view, an activity diagram for our process view, and a component diagram for our physical view. These are necessary components of the 4+1 view. • Added labels to Requirements tables, allowing for simpler naming and references of requirements.
11/14/2025	RD, RT, CV: • Added User Interface Design, Logical View, Development View, Process View, and Physical View chapters, representing the 4+1 view of the FindIt system. • Added manual versioning for documentation (for now). • Added Figma Paper Prototype example for the FindIt user interface
11/07/2025	RD, RT, CV: • Added Weekly Report 10 • Dynamically referenced requirements in Requirements Chapter.
10/31/2025	RD, RT, CV: • Added Weekly Report 9 • Added more activity diagrams to replicate the flows of our use cases

Table 1: Document Update History

Date	Updates
10/24/2025	RD, RT, CV: • Added Development Notes to appendix to track and log any specific processes or findings that we may come across throughout our development. • Mapped Requirements to use cases, and added references to user stories. • Added Weekly Report 8
10/17/2025	RD, RT, CV: • Added Use Cases Chapter, including: – Table of Use Cases with all six primary use cases. – Use Case Diagram. – Detailed Use Case Descriptions for each primary use case. – System Activity Diagram • Added Weekly Report 7
10/10/2025	RD, RT, CV: • Added Requirements Chapter • Added User Stories Chapter. • Added Use Cases Chapter. • Updated Glossary • Added Weekly Report 6
10/02/2025	RD, RT, CV: • Added Development Plan chapter.
09/30/2025	RD, RT, CV: • Added Bibliography to appendix • Moved Weekly Reports to appendix.
09/25/2025	RD, RT, CV: • Added Weekly Report 4
09/18/2025	RD, RT, CV: • Edited and Finished Team Declaration chapter • Added Weekly Report 3
09/16/2025	RD, RT, CV: • Added Team Declaration chapter
09/12/2025	RD, RT, CV: • Added Roy Tas's introduction • Added Weekly Report 2 • Created System overview diagram
09/04/2025	RD, RT, CV: • Added introduction page, introducing the team. • Added Weekly Report 1

Table of Contents

List of Tables

List of Figures

Chapter 1

Introduction

- Ryan Davis, Cory Vitanza, and Roy Tas

My name is Ryan Davis, and I am a Software Engineering major. I am in my senior year of study pursuing my bachelor's degree. I hope to continue my Software Engineering career one day as the founder of a startup company.

Hi, my name is Cory Vitanza, a 4th-year Software Engineering student also currently pursuing a Master's in Engineering Management. I've gained coding experience from my classes and real-world experience in QA and testing during my internship at Trimble MAPS. After my time here at Stevens, I plan to continue working in QA, hopefully in a management position soon enough. My favorite coding language is C++, and I love working on challenging projects. Outside of coding, I'm into music, cars, video games, and watching / playing sports, especially basketball and football

My name is Roy Tas I am a senior Engineering Management major concentrating in Financial Engineering. I have some C++ and Python experience but I will be focusing more on the scalability and logistics of our project. Post-graduation I want to pursue a career in jewelry manufacturing.

Chapter 2

Team Declaration

- Ryan Davis, Cory Vitanza, and Roy Tas

2.1 Team Information

- **Team Name:** FindIt
- **Team Members:**
  1. Ryan Davis
  2. Cory Vitanza
  3. Roy Tas

2.2 Project Proposal

Patrons with disabilities such as autism or conditions such as dementia tend to wander or get lost, causing families and caregivers of such patrons great amounts of stress. We are FindIt and we aim to mitigate stress, with our new tracking solution employing LoRaWAN (Long Range Wide Area Network) technology to track locations at far distances, with extensive battery life.

Our device aims to demonstrate that a low-cost and accessible solution can balance affordability, reliability, and ease of use- as compared to current tracking solutions like AirTags or other commercial tracking devices. We envision FindIt as a scalable option that extends beyond families to institutional applications such as nursing homes, group homes, and schools, where safety and monitoring are critical. Our device comforts families with efficient tracking tools to ensure comfort when their loved ones are lost.

2.3 Mission Statement

The mission of FindIt is to design and deliver a reliable, affordable, and user-friendly location tracking solution for families and caregivers of individuals at risk of wandering. Our goal is to reduce stress by providing a reliable tool that allows quick and efficient tracking in critical situations. The business need is the lack of affordable long-range tracking technology for families,

and the business intent is to bridge that gap by leveraging LoRaWAN for extended battery life and coverage.

2.4 Key Drivers

Several key drivers motivate the development of this solution.

There is a great need for improved tracking solutions for wandering patrons. Wandering is prevalent amonguch groups with autism or dementia, and existing solutions are expensive, insufficient, and at times inaccurate. We aim to minimize these issues with the development of our device.

Current solutions for asset tracking, like AirTags or cellular-based trackers, can become expensive for long-term usage. Using LoRaWAN protocols, our device reduces recurring costs and enhances battery life.

Technological accessibility is also an important motivator, as our group aims to reduce the stress of figuring out an application when time is of the essence. Our service is also subscription-free, in order to benefit all those in need.

LoRaWAN's versatility allows for uses beyond families. Group facilities such as schools, assisted living centers, or hospitals could benefit from a low-cost, universal system to monitor the safety of many individuals simultaneously.

2.5 Key Constraints

- **Network Coverage:** LoRaWAN's effectiveness depends on gateway proximity. Limited coverage in some areas may impact reliability, requiring design options for sparse networks. LoRaWAN coverage maps detail that Urban areas, specifically in the Northeast, have excellent coverage.
- **Battery Life vs. Device Size:** Maintaining strong battery life while preserving the small size of this device is optimal for the constant carriage of our unit. The device must remain lightweight and comfortable for continuous use.
- **Privacy and Security:** Storing and sending location data leads to privacy concerns. Secure data handling, LoRaWAN encryption, and user-controlled permissions are necessary to maintain trust with the user.
- **Cost Targets:** Affordability is a primary selling point. Hardware, manufacturing, and deployment costs must remain competitive without sacrificing reliability.
- **Interface Simplicity:** Achieving a clean, intuitive interface while maintaining accessibility features is critical for the tracking unit to reach diverse user bases with differing technological expertise.

Chapter 3

Development Plan

- Ryan Davis, Cory Vitanza, and Roy Tas

3.1 Introduction

The purpose of this project is to design, implement, and validate a location-tracking and data communication system using LoRaWAN-enabled hardware integrated with a cloud-based software platform. The system aims to provide reliable long-range wireless data transmission between embedded devices and a backend service, with supporting web-based tools for monitoring and management. By combining embedded development, cloud infrastructure, and a user-facing interface, this project demonstrates the end-to-end development process of a distributed system.

Success criteria for the project include (1) establishing reliable LoRaWAN communication between the embedded board and backend, (2) ensuring data is properly stored, processed, and displayed through the frontend application, and (3) passing acceptance testing with the Wearable Robotics Systems Lab. From a user perspective, the system enables simple, reliable collection and visualization of data in real time, serving as a proof-of-concept platform that can be extended to future applications.

3.2 Roles and Responsibilities

Clear definition of roles and responsibilities ensures accountability and effective collaboration within the team. Since this project is being developed by a small group, several members will serve multiple roles across development, testing, documentation, and risk management. The following assignments identify primary responsibilities, while recognizing that all team members will contribute collaboratively to design decisions, implementation, and review processes.

1. Development Lead: Ryan
2. Buildmeister: Roy
3. Architect: Cory
4. Developers: Ryan (Backend), Cory (Frontend), Roy (Infrastructure)

5. Test Lead: Cory
6. Testers: All
7. Documentation: All
8. Documentation Editor: All
9. Designer: Cory
10. User advocate: Ryan Davis
11. Risk Management: Roy
12. System Administrator: Ryan
13. Modification Request Board: All
14. Requirements Resource: Roy
15. Customer Representative: All
16. Customer responsible for acceptance testing: Wearable Robotics Systems Lab

3.3 Method

These are unique to software development, although there may be some overlap.

3.3.1 Software

1. Language(s):
    - C (Arduino IDE)
    - Python (3.x)
    - TypeScript (React)
    - SQL (PostgreSQL via Supabase)
2. Operating System(s):
    - Arduino embedded OS (board-specific)
    - Linux/Ubuntu (for Docker and AWS deployment)
    - macOS/Windows (development environments)
3. Software packages/libraries used:
    - LoRaWAN protocol (TCP/IP stack integration)
    - Supabase (PostgreSQL backend service)

- Google Maps API
- React (TypeScript frontend framework)
- Docker (containerization)
- AWS (deployment and hosting)
- GitHub (version control)
- Kanban Board (Agile task management)

4. Code conventions:

- GitHub repository guidelines for commits and pull requests
- ESLint + Prettier for TypeScript (frontend)
- PEP 8 for Python (backend tools)
- Arduino C style conventions

3.3.2 Hardware

1. Development Hardware

- Wio Tracker 1110 Development Board
- Semtech LR1110 LoRa® transceiver

2. Test Hardware

- Wio Tracker 1110 Development Board
- LoRaWAN Gateway (connected to The Things Stack)

3. Target/Deployment Hardware

- Wio Tracker 1110 Development Board
- Semtech LR1110 LoRa® transceiver
- LoRaWAN Gateway (The Things Stack integration for live deployment)

3.3.3 Backup plan (individual and project)

In case the Wio Tracker is not capable of the deployment of our software, the FindIt team will need to research and develop board alternatives. One option for this scenario would be to use a Raspberry Pi Nano, implemented with an external LoRaWAN module to allow for data transmission. Even with such addition, it is necessary for our unit to maintain battery consumption amongst boards, ensuring our unit has extensive battery life for users.

3.3.4 Review Process

1. Will you do architecture, usability, design, security, privacy or code reviews?
   - Code reviews will be completed upon the completion of Agile tasks on our Kanban board.
2. What approach will you use for the reviews (formal, informal, corporate standard)?
   - Informal reviews will be employed to achieve the approval of all members of the team, ensuring that all cases are handled in new implementation.
3. Who is responsible for the reviews and resolving any issues uncovered by the reviews?
   - Upon the implementation of a component, the other two members of the team will perform a code review. Any issue uncovered from code review will be resolved by the member working on the component.

3.3.5 Build Plan

1. Revision control system and repository used
   - GitHub (Version Control)
2. Continuous integration
   - Circle CI
3. Deadlines for the builds
   - Depending on our planned sprints, our Kanban board will present deadlines based on tasks.
4. Regression test process – see test plan

3.3.6 Modification Request Process

1. MR tool: For modification requests, we will use our Kanban board on GitHub to issue any modification requests as a task in the board.
2. Decision process: Decisions for modification will be decided during heartbeat meetings, after a discussion among the team concerning the modification. The team will proceed to assign a task to the Kanban board on the methodology and developers necessary for the task.
3. State whether there will be two process streams one during development and one after development:
   - During development, there will be two process streams. One is this programming of the development board and the other will be for the front-end user interface. The two streams will be integrated for deployment.

3.4 Virtual and Real Workspace

3.4.1 Virtual Workspace

Our team will utilize texts for direct communication, as well as a Discord server used for the sharing of files, resources, and communication. GitHub will also be used for secondary communication, implementing comments and changes to the code base via commits and merge requests.

3.4.2 Real Workspace

The team's regular workspace will be the Software Engineering Lab, primarily for heartbeat meetings and the development of our program. Meetings with Dr. Zanotto will be held in his office or over Zoom, when necessary.

3.5 COMMUNICATION PLAN

3.5.1 Heartbeat Meetings

Heartbeat meetings will be held at least twice per week with all group members present. These meetings serve to take the "pulse" of the project by providing quick updates, raising blockers, and reviewing open issues and risks. Each meeting will follow a structured agenda that includes:

- Individual progress updates
- Discussion of current blockers or risks
- Review of action items and open issues

Notes from each meeting will be recorded in the project document, and the GitHub Kanban board will be updated accordingly. These meetings are designed to be short, ideally during our allotted class-time.

3.5.2 Status Meetings

Status meetings will occur once a month and will include the project group and Dr. Damiano Zanotto. The purpose of these meetings is to present overall project progress, demonstrate completed work, and receive feedback on areas for improvement. If issues arise during these meetings, they will be addressed separately in an issues meeting. These meetings are intended to be concise, focusing on high-level updates and progress checkpoints.

3.5.3 Issues Meetings

Issues meetings will be scheduled whenever significant problems arise that require group consensus or immediate attention. Alerts for these meetings will typically be raised through a Discord message. If consensus is reached that a meeting is necessary, it will be scheduled as soon as all

group members are available. These meetings provide a dedicated space for problem-solving and decision-making outside of the regular heartbeat meetings.

3.6 Timeline and Milestones

The following milestones outline the major deliverables for the project. Each milestone represents a 100% complete item, with critical participants, begin and end dates, and deliverables clearly defined.

- **Milestone 1: Project Setup & Requirements Finalization**
  - Begin: Oct 3, 2025 End: Oct 17, 2025
  - Deliverables: Project repository initialized, Kanban board configured, requirements document finalized, hardware/software resources received and confirmed.
- **Milestone 2: Core System Architecture & Prototype Communication**
  - Begin: Oct 17, 2025 End: Oct 31, 2025
  - Deliverables: Draft system architecture, initial Arduino board programming for basic LoRaWAN communication, mock frontend interface, database schema in Supabase.
- **Milestone 3: Integration of Hardware and Software Components**
  - Begin: Oct 31, 2025 End: Nov 14, 2025
  - Deliverables: Hardware successfully transmitting data to backend, frontend connected to database, Docker setup for local deployment, integration testing initiated.
- **Milestone 4: Feature Implementation & System Testing**
  - Begin: Nov 14, 2025 End: Nov 28, 2025
  - Deliverables: Full set of planned features implemented, CI/CD pipeline operational, unit + integration tests passing, preliminary user documentation drafted.
- **Milestone 5: Final Review & Delivery**
  - Begin: Nov 28, 2025 End: Dec 12, 2025
  - Deliverables: Final acceptance testing completed, documentation finalized, project demonstration delivered, final repository and deployment package prepared for submission.

This document will be re-issued at each milestone to reflect progress and highlight changes since the previous revision.

3.7 Testing Policy/Plan

Our testing policy combines automated and manual approaches to ensure that the project meets functional requirements and maintains high reliability. The following types of testing will be performed:

- **Unit Testing:** We will implement unit tests for critical components, including geofence logic and location-based features. Pytest will be the primary testing framework. Unit testing will begin as soon as core modules are developed and will continue throughout the project.
- **Integration Testing:** Integration testing will verify interactions between different system components, such as API calls to The Things Stack and web service calls to the Google Maps API. These tests will be mocked where appropriate to ensure reliability and repeatability.
- **System Testing:** End-to-end system testing will be conducted once a minimum viable product is available. This will include route pings, location checking, and verifying that hosted meals can be created, displayed, and joined.
- **Acceptance Testing:** Acceptance testing will be performed in collaboration with Dr. Damiano Zanotto. This testing will validate that the system meets the agreed project requirements and is ready for delivery.
- **Regression Testing:** Regression tests will be rerun whenever bug fixes or new features are introduced to ensure that existing functionality remains stable.
- **Testing for Quality Attributes:** While performance and security are not the primary focus, geolocation accuracy and reliability will be emphasized. Specific attention will be given to ensuring that geofences trigger consistently and API interactions are fault-tolerant.
- **Continuous Integration (CI):** CircleCI will be used to automate testing during development. All code changes will trigger automated test runs to catch issues early.

Our stopping criteria for testing is defined as achieving at least 80% unit test coverage across critical modules, successful integration of geofencing and API functionality, and passing acceptance testing with project stakeholders.

3.8 Risks

Risks that we must mitigate throughout the development of our product include various issues on all scales of development. With the development board, we want to ensure that the battery life is competitive with existing tracking units, such as AirTag. We also would like to ensure that the tracking unit is able to provide precise location data for the user to accurately locate the unit.

3.9 Assumptions (required)

For the development of our program, it is assumed that:

- Publicly accessible LoRaWAN gateways populate our region, allowing for fluid connection between our board to LoRaWAN network servers.
- End users have mobile devices or computers that are able to access our website, providing them access to their tracking units.

3.10 DISTRIBUTION LIST

This document will be shared with the members of the FindIt team, as well as Dr. David Darian Muresan and Dr. Damiano Zanotto throughout our development processes.

3.11 IRB Protocol

Our project does not require IRB protocol.

3.12 Required Resource and Budget

3.12.1 Hardware Resources

For our tracking unit, the only development expense will include the Wio Tracker 1110 development board, costing $30.

3.12.2 Software Resources

Depending on the size of our infrastructure, deployment on AWS as well as the storage of historical data on Supabase will require subscriptions in order to maintain the usage of our software.

Chapter 4

Requirements

– Ryan Davis, Cory Vitanza, and Roy Tas

4.1 Stakeholders

- Wearable Robotics Systems Lab
- Assisted Living Homes
- Patrons with autism or dementia

4.1.1 Customers

- Families with autistic members
- Caregivers for dementia patients
- Assisted living homes

4.1.2 Sponsors

- Wearable Robotic Systems Lab
- Stevens Institute of Technology

4.1.3 Competitors

Device	**Price & Subscription**	**Connection Type**	**Target Users**	**Key Features**
Angel Sense	$220 + $45/month	Cellular	Autism / Children	Guardians, alerts & geofences, live safe ride transit tracking
Jiobit Smart Tag	$140 + $10/month	Cellular	Elderly, children, autism	Location tracking
Medical Guardian	$150 + $40/month	Cellular	Elderly	SOS button directly connecting to emergency operators, fall detection

Table 4.1: Comparison of Elderly and Autism Tracking Devices

4.2 Key Concepts

- LoRaWAN (Long Range Wide Area Network) enables long-range, low-power communication.
- GPS (Global Positioning System) provides location coordinates for tracker position.
- Wio Tracker 1110 serves as the hardware platform.
- Supabase stores GPS data and user configurations.
- RSSI (Received Signal Strength Indicator) measures LoRaWAN signal strength.
- Device Uplink sends GPS data to the backend.
- Device Downlink updates device configuration remotely.
- Gateway receives device transmissions.
- The Things Stack (TTS) (*The Things Stack*) receives payloads and forwards them to Supabase.

4.3 User Requirements

Table 4.2: User Requirements Table

Requirement	Priority	Use Case(s)
FR1 (`reqfGeofence`): The system shall support geofencing with alerts when a tracker exits predefined boundaries.	MustHave	ucGeofenceAlert
FR2 (`reqfRealtimeUpdate`): The system shall provide real-time location updates via the caregiver mobile app.	MustHave	ucGeofenceAlert
FR3 (`reqfMultiDevice`): The system shall support simultaneous monitoring of multiple trackers.	MustHave	ucMultiResidentMonitoring
FR4 (`reqfConfigNotifications`): The system shall allow configurable notifications for multiple users or devices.	ShouldHave	ucMultiResidentMonitoring
FR5 (`reqfBatteryMonitor`): The device shall monitor remaining battery capacity and send low-power alerts.	MustHave	ucBatteryStatusAlert
FR6 (`reqfBatteryLife`): The system shall maintain battery life comparable to consumer devices such as Apple AirTag.	CouldHave	ucBatteryStatusAlert
FR7 (`reqfEmergencyButton`): The system shall include a button that triggers an emergency alert notification to caregivers.	ShouldHave	ucEmergencyAlert
FR8 (`reqfPatternDetection`): The system shall use a machine learning algorithm to detect abnormal movement patterns.	ShouldHave	ucPatternDetection
CR1 (`reqcNoSubscription`): The system shall operate without subscription fees by using community LoRaWAN gateways.	MustHave	ucCostEfficientDeployment
QR1 (`reqqScalability`): The system shall scale to support multiple concurrent devices and users.	ShouldHave	ucCostEfficientDeployment
QR2 (`reqqModelPerformance`): The ML model shall maintain accuracy in detecting anomalies without degrading battery life.	CouldHave	ucPatternDetection

4.4 System (Constraints) Requirements

Table 4.3: System (Constraints) Requirements

Requirement	Priority	Type
SR1 (`reqsConnectivity`): The system shall connect to at least one LoRaWAN gateway within a 2 km radius under normal conditions.	MustHave	Functional
SR2 (`reqsTransmission`): The tracker shall send GPS coordinates via uplink every configurable interval between 1–10 minutes.	MustHave	Functional
SR3 (`reqsAPI`): Supabase shall provide a secure RESTful API for data storage and retrieval.	MustHave	Functional
SR4 (`reqsBattery`): The tracker shall report battery status with each payload.	ShouldHave	Functional
SR5 (`reqsEmergency`): The emergency button shall send an uplink event within 5 seconds of activation.	MustHave	Functional
SR6 (`reqsConfig`): Downlink commands shall allow caregivers to modify device settings remotely.	ShouldHave	Functional
SR7 (`reqsSecurity`): All communications between Supabase and TTS shall use encrypted HTTPS endpoints.	MustHave	Security
SR8 (`reqsRetention`): Supabase shall store location history data for at least 30 days.	ShouldHave	Functional
SR9 (`reqsOffline`): If no gateway is available, the tracker shall cache up to 10 unsent messages until reconnection.	ShouldHave	Reliability

4.5 Non-functional (Quality) Requirements

Table 4.4: Non-functional (Quality) Requirements

Requirement	**Priority**	**Category**
NFR1 (`reqnPerformance`): Uplink latency between device transmission and database update shall not exceed 15 seconds.	MustHave	Performance
NFR2 (`reqnUsability`): The caregiver app shall display alerts within two screen interactions.	ShouldHave	Usability
NFR3 (`reqnMaintainability`): The system shall be modular, allowing firmware or API updates independently.	ShouldHave	Maintainability
NFR4 (`reqnSecurity`): All stored user data shall follow AES-256 encryption standards.	MustHave	Security
NFR5 (`reqnPower`): The device shall operate for at least 5 days on a single charge at 15-minute intervals.	ShouldHave	Efficiency

4.6 Domain (Business) Requirements

Table 4.5: Domain (Business) Requirements

Requirement	**Priority**	**Type**
BR1 (`reqbCompliance`): The system shall comply with HIPAA regulations if identifiable patient data is stored.	MustHave	Legal
BR2 (`reqbOpenInfra`): The project shall rely on public LoRaWAN gateways instead of proprietary cellular networks.	ShouldHave	Business
BR3 (`reqbOwnership`): All user data shall remain the property of the caregiver or family account holder.	MustHave	Policy
BR4 (`reqbMaintenance`): Hardware upkeep shall be the responsibility of the owning institution or family.	ShouldHave	Operational

4.7 Requirements to Use Case Mapping

Table 4.6: Requirements to Use Case Mapping

Requirement ID	Requirement Description	Mapped Use Case(s)
FR1	Geofence detection and alerts.	ucGeofenceAlert
FR3	Multi-device tracking for caregivers.	ucMultiResidentMonitoring
FR5	Battery alerts for device monitoring.	ucBatteryStatusAlert
CR1	No subscription-based infrastructure.	ucCostEfficientDeployment
FR7	Emergency button integration.	ucEmergencyAlert
FR8	ML anomaly detection of movements.	ucPatternDetection

4.8 Traceability Matrix

Table 4.7: Requirements, Use Case, and User Story Traceability Matrix

Req ID	Use Case(s)	User Story(ies)	Description
FR1	ucGeofenceAlert	usParentAutism	Geofence alert notifications.
FR3	ucMultiResidentMonitoring	usCaregiver	Multi-resident caregiver monitoring.
FR5	ucBatteryStatusAlert	usBattery	Battery monitoring alerts.
FR7	ucEmergencyAlert	usEmergency	Emergency button response.
CR1	ucCostEfficientDeployment	usSysAdmin	Cost-efficient infrastructure.
QR2	ucPatternDetection	usAIPattern	ML anomaly detection accuracy.

Chapter 5

User Stories

– Ryan Davis, Cory Vitanza, and Roy Tas

5.1 Overview

This chapter presents key user stories that describe the real-world needs and motivations of potential users of the tracking system. These stories help guide the system's design and ensure that functional and non-functional requirements align with user expectations.

5.2 User Stories

5.2.1 Parent of a Child with Autism

- **US1**: **As a parent of a child with autism,** I want a device that automatically alerts me when my child leaves a safe zone so that I can respond quickly without wasting time searching.

5.2.2 Caregiver in an Assisted Living Facility

- **US2**: **As a caregiver responsible for multiple residents,** I want to track multiple devices simultaneously and receive alerts when a resident leaves a designated safe area so that I can act promptly to ensure their safety.

5.2.3 Family Member Monitoring Battery Life

- **US3**: **As a family member monitoring a loved one,** I want to receive notifications when the tracker's battery is running low so that I can prevent the device from powering off unexpectedly during an emergency.

5.2.4 User Triggering an Emergency Alert

- **US4**: **As a user wearing the device,** I want to press a button in an emergency to send an alert to my caregiver or family members so that I can get help quickly.

5.2.5 System Administrator Managing Devices

- **US5**: **As a system administrator,** I want to manage LoRaWAN gateways and device registrations efficiently so that the system remains scalable and reliable without subscription costs.

5.2.6 AI Monitoring and Pattern Detection

- **US6**: **As a system backend operator,** I want the system to automatically detect unusual movement patterns so that potential emergencies can be identified proactively.

5.3 User Story to Use Case Mapping

Table 5.1: User Story to Use Case Mapping

User Story ID	Description	Mapped Use Case(s)
usParentAutism	Parent receives alerts when a child leaves a safe zone.	ucGeofenceAlert
usCaregiver	Caregiver monitors multiple residents and receives alerts.	ucMultiResidentMonitoring
usBattery	Family member receives low-battery notifications.	ucBatteryStatusAlert
usEmergency	User triggers an emergency alert to notify caregiver.	ucEmergencyAlert
usSysAdmin	System administrator manages deployments and gateways.	ucCostEfficientDeployment
usAIPattern	Backend detects unusual behavior through ML models.	ucPatternDetection

5.4 Traceability Discussion

All user stories directly support at least one system use case to ensure end-to-end traceability. Each story reflects the needs of a real stakeholder—caregivers, users, family members, or system administrators—and guides the functional and non-functional requirements of the FindIt asset tracking system. By maintaining explicit links (usParentAutism–usAIPattern) between stories and use cases, this document ensures future system iterations remain aligned with user needs.

Chapter 6

Use Cases

– Ryan Davis, Roy Tas, Cory Vitanza

This chapter presents the use case diagrams that satisfy the requirements of the FindIt asset tracking system. Each use case represents a key user interaction scenario that drives the system requirements. All use cases are mapped to at least one requirement to ensure complete traceability.

6.1 Table of Use Cases

Table 6.1: Use Cases Table

Use Case	Requirements	Name and Description
ucGeofenceAlert	FR1, FR2	ucGeofenceAlert: **Geofence Alert** – The system detects when the tracker exits or enters a predefined boundary and sends an alert to the caregiver's phone.
ucMultiResidentMonitoring	FR3, FR4	ucMultiResidentMonitoring: **Multi-Resident Monitoring** – A caregiver monitors multiple trackers simultaneously and receives notifications when any resident leaves a safe area.
ucBatteryStatusAlert	FR5, FR6	ucBatteryStatusAlert: **Battery Status Alert** – The tracker measures its remaining battery and sends a notification when power is low.
ucCostEfficientDeployment	CR1, QR1	ucCostEfficientDeployment: **Cost-Efficient Deployment** – The system operates without subscription costs, relying on LoRaWAN gateways, and supports scaling across many devices.
ucEmergencyAlert	FR7, ??	ucEmergencyAlert: **Emergency Alert** – A user can trigger an alert in case of a fall or emergency, notifying caregivers or family members for immediate response.
ucPatternDetection	FR8, QR2	ucPatternDetection: **Machine Learning Pattern Detection** – The system analyzes historical tracker data using a machine learning model to identify unusual movement or location patterns, alerting the caregiver or user interface of potential anomalies.

6.2 Use Case Diagrams

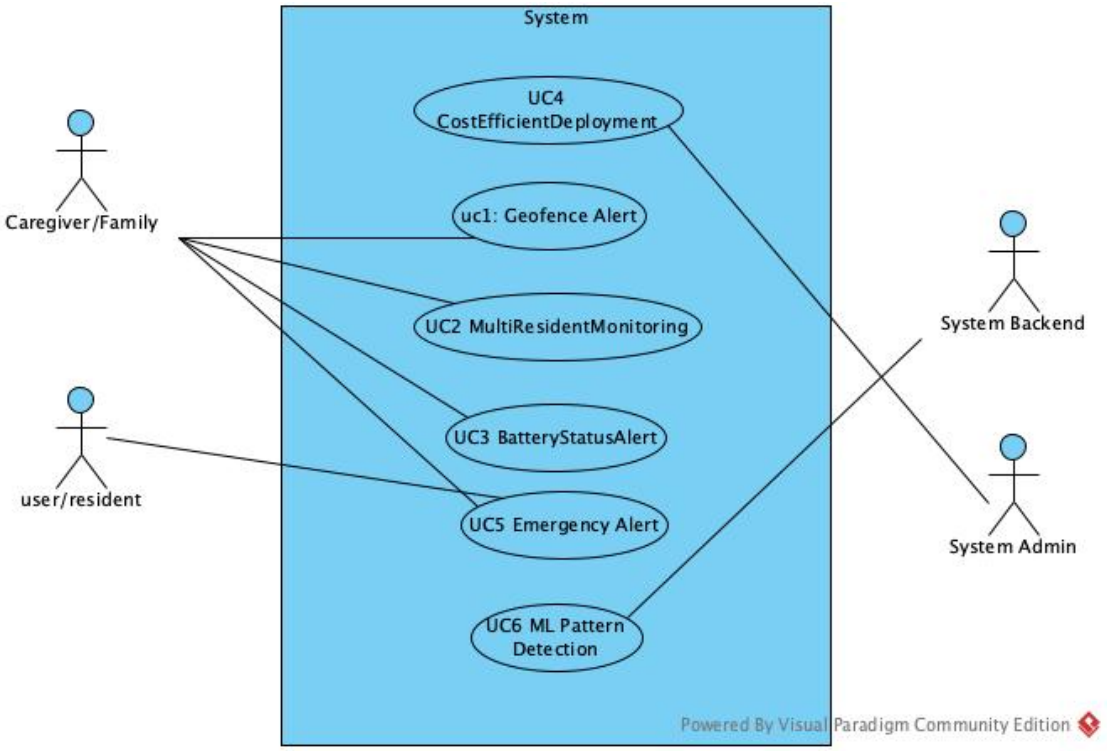

Figure 6.1: finditUseCases: finditUseCases FindIt system use case diagram, showing references to primary use cases.

6.3 Use Case Descriptions

Table 6.2: ucGeofenceAlert. Geofence Alert

Use Case UC-6.1 (ucGeofenceAlert)**.** ucGeofenceAlert **Geofence Alert** – Detects when the tracker exits a predefined area and notifies the caregiver.
Requirements: **FR1**, **FR2**
Primary actors: Caregiver, Tracker Device
Preconditions: Tracker and caregiver app are connected to the LoRaWAN network.
Main flow: 1. The caregiver defines a geofence area. 2. The tracker transmits periodic GPS data. 3. The system detects an exit from the geofence. 4. A notification is sent to the caregiver.
Postconditions: Caregiver receives alert in near-real time.

Table 6.3: ucMultiResidentMonitoring. Multi-Resident Monitoring

Use Case UC-6.2 (ucMultiResidentMonitoring)**.** ucMultiResidentMonitoring **Multi-Resident Monitoring** – Allows caregivers to view and track multiple residents simultaneously.
Requirements: **FR3**, **FR4**
Primary actors: Caregiver
Preconditions: Multiple FindIt trackers are registered to one caregiver account.
Main flow: 1. The caregiver logs into the FindIt dashboard. 2. Multiple devices report position updates. 3. The caregiver views the dashboard for status of all trackers.
Postconditions: The caregiver can identify any tracker that leaves its assigned safe zone.

Table 6.4: ucBatteryStatusAlert. Battery Status Alert

Use Case UC-6.3 (ucBatteryStatusAlert)**.** ucBatteryStatusAlert **Battery Status Alert** – Sends low battery notifications to caregivers.
Requirements: **FR5**, **FR6**
Primary actors: Tracker Device, Caregiver
Main flow: 1. Tracker measures remaining battery capacity. 2. Device sends low battery alert to the caregiver app. 3. Caregiver replaces or recharges the tracker.
Postconditions: Caregiver receives timely low battery alerts to maintain uptime.

Table 6.5: ucCostEfficientDeployment. Cost-Efficient Deployment

Use Case UC-6.4 (ucCostEfficientDeployment)**.** ucCostEfficientDeployment **Cost-Efficient Deployment** – Uses community LoRaWAN networks and low cost backend services to reduce ongoing expenses.
Requirements: **CR1**, **QR1**
Primary actors: System Administrator, Caregiver
Main flow: 1. Tracker connects through community LoRaWAN gateway. 2. Data stored in Supabase backend. 3. System supports additional users with minimal subscription fees.
Postconditions: The system remains cost-efficient and scalable.

Table 6.6: ucEmergencyAlert. Emergency Alert

Use Case UC-6.5 (ucEmergencyAlert)**.** ucEmergencyAlert **Emergency Alert** – Allows users to trigger an emergency notification.
Requirements: **FR7**, **??**
Primary actors: User, Caregiver
Preconditions: Tracker device powered and connected.
Main flow: 1. The user presses the emergency button. 2. Tracker sends an emergency alert to the caregiver app. 3. (Optional) Additional redundancy mechanisms may be added in future versions.
Postconditions: Caregiver receives emergency alert promptly.

Table 6.7: ucPatternDetection. Machine Learning Pattern Detection

Use Case UC-6.6 (ucPatternDetection)**.** ucPatternDetection **Machine Learning Pattern Detection** – The system uses trained algorithms to detect deviations in a user's movement or daily routine.
Requirements: **FR8**, **QR2**
Primary actors: Caregiver, System Backend (AI Engine)
Preconditions: The tracker has collected sufficient historical location data for the ML model to learn baseline patterns.
Main flow: 1. The tracker periodically uploads GPS and activity data to the backend. 2. The ML engine trains on location history and identifies regular movement patterns. 3. The system detects deviations (e.g., unexpected detours, prolonged immobility). 4. An alert is sent to the caregiver through the user interface.
Postconditions: Abnormal movement or potential emergencies are detected early and communicated to the user interface.

6.4 Activity Diagrams

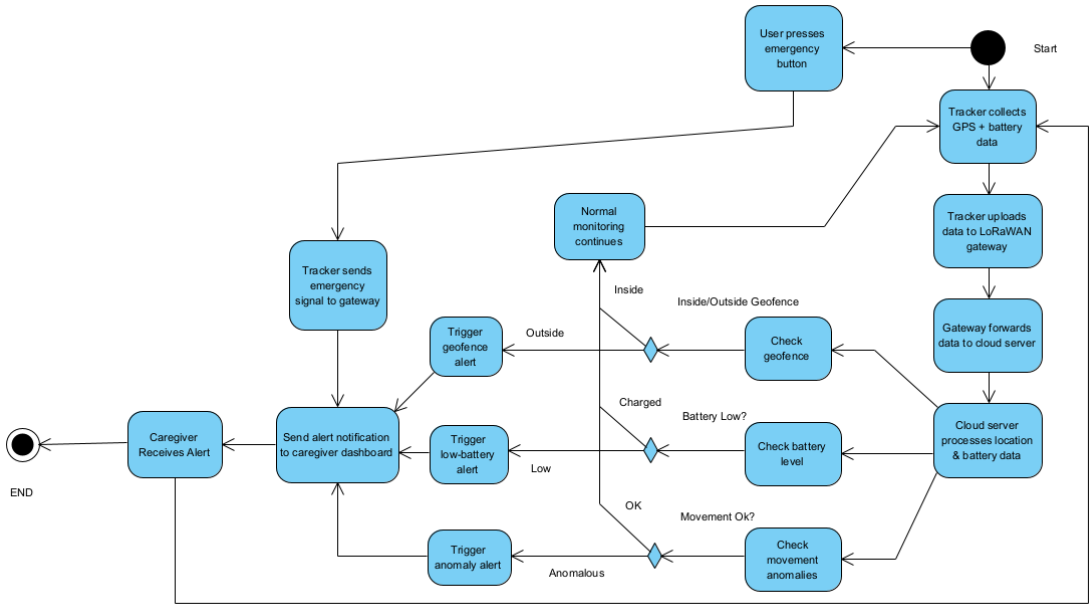

Figure 6.2: finditActivity1: finditActivity1 Overall system activity diagram for the FindIt tracking system, illustrating the continuous monitoring loop, data processing, and alert notification flows between the tracker, cloud, and caregiver.

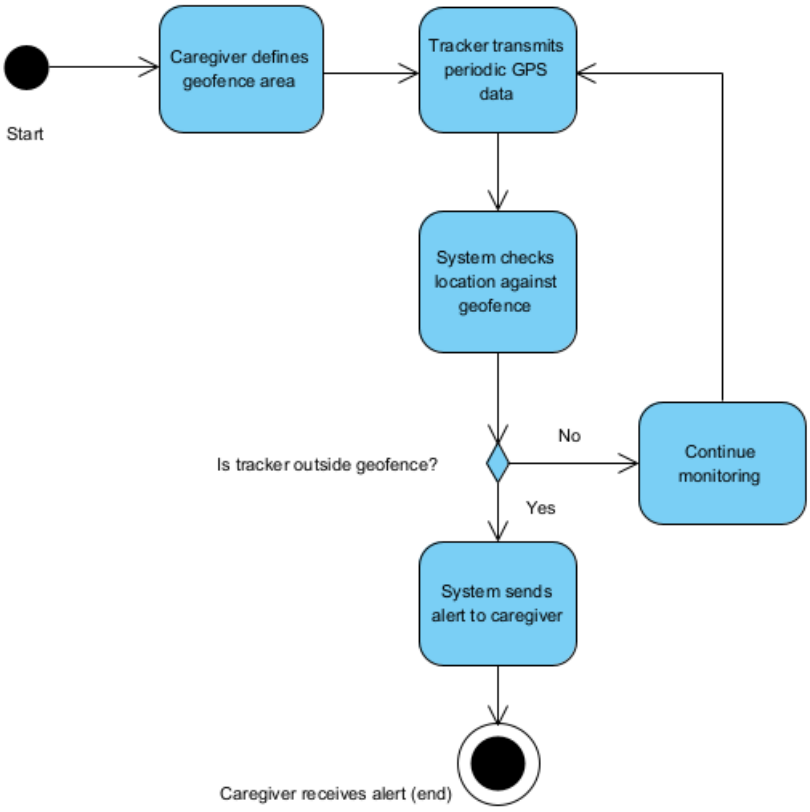

Figure 6.3: ucGeofenceAlertActivity: Activity Diagram for Geofence Alert (UC1)

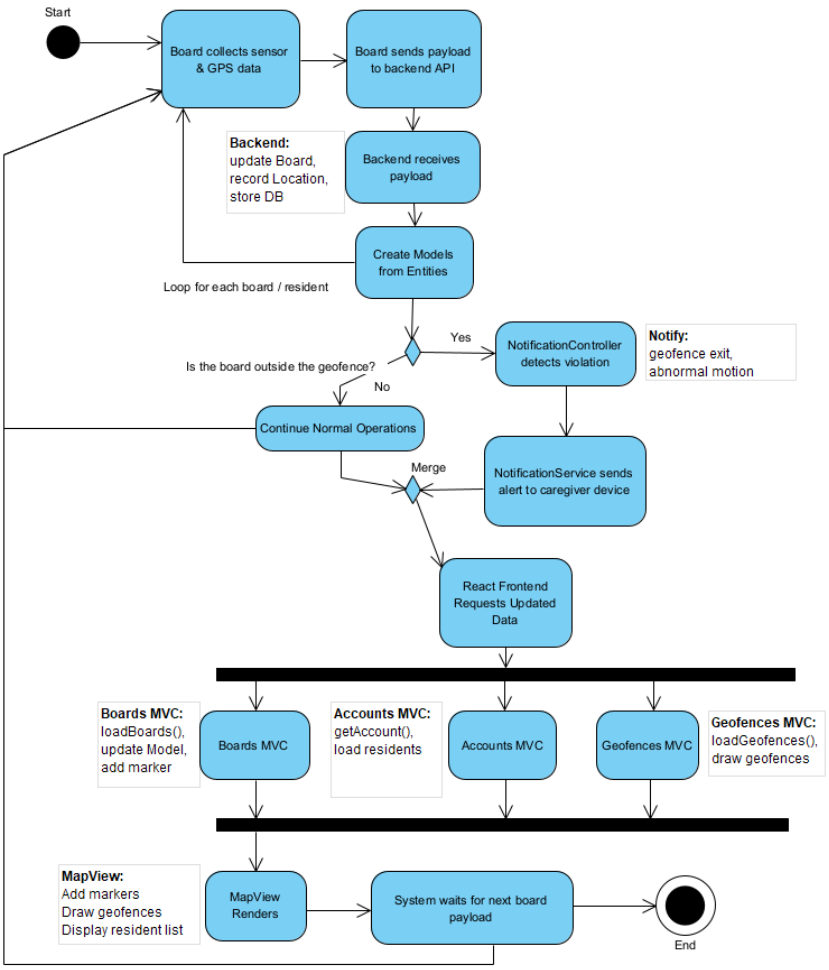

Figure 6.4: ucMultiResidentMonitoringActivity: Activity Diagram for Multi-Resident Monitoring (UC2)

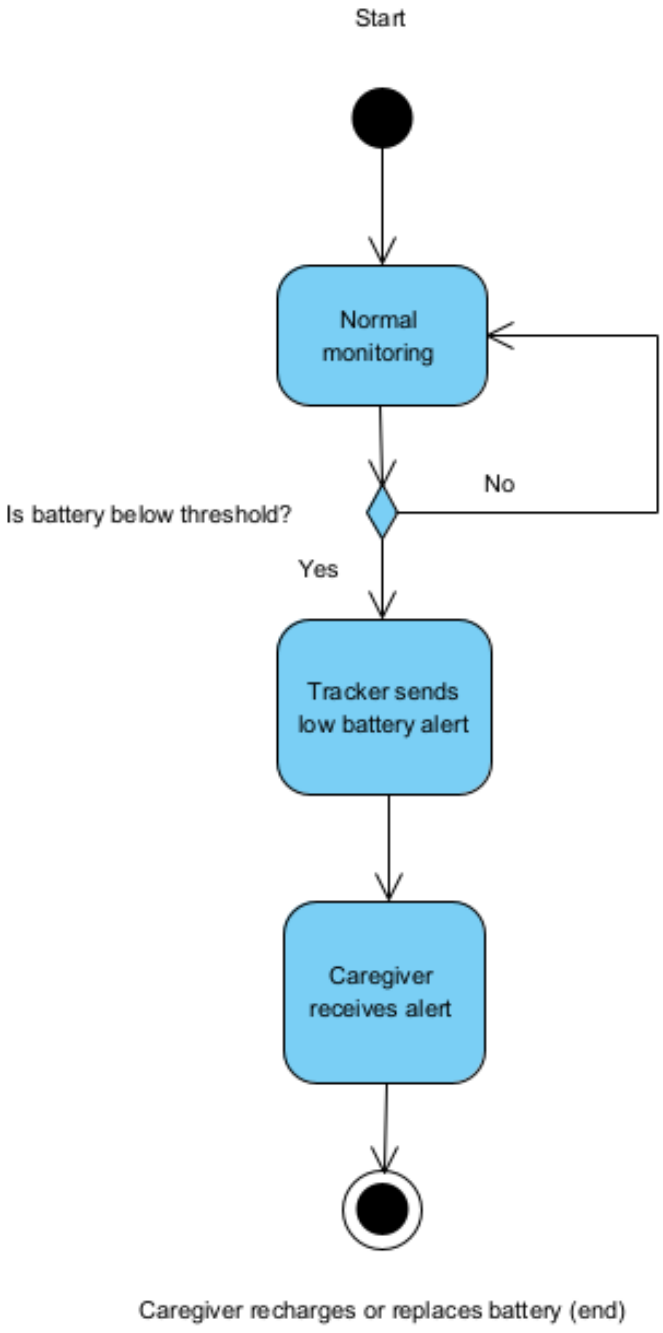

Figure 6.5: ucBatteryStatusAlertActivity: Activity Diagram for Battery Status Alert (UC3)

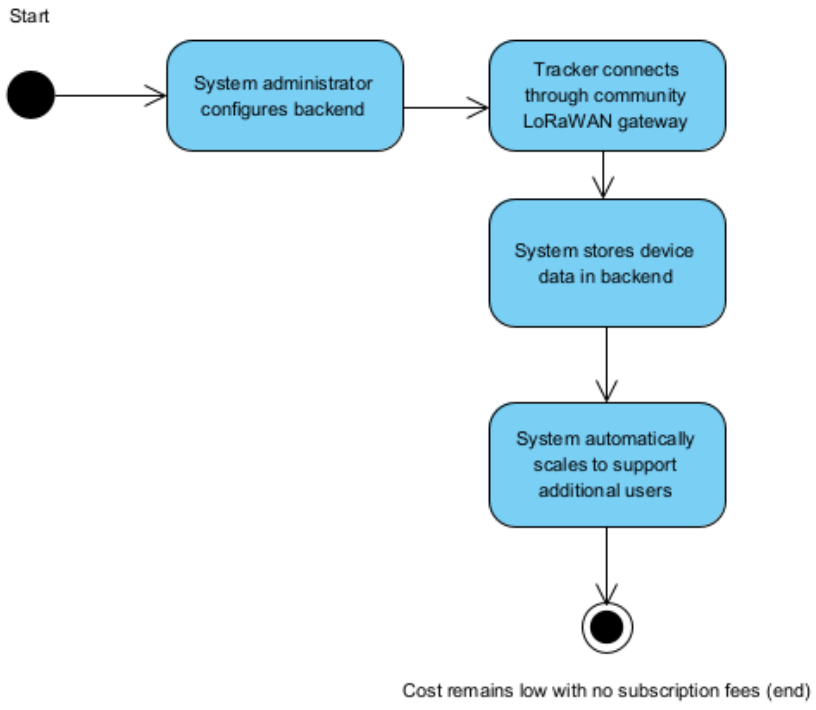

Figure 6.6: ucCostEfficientDeploymentActivity: Activity Diagram for Cost-Efficient Deployment (UC4)

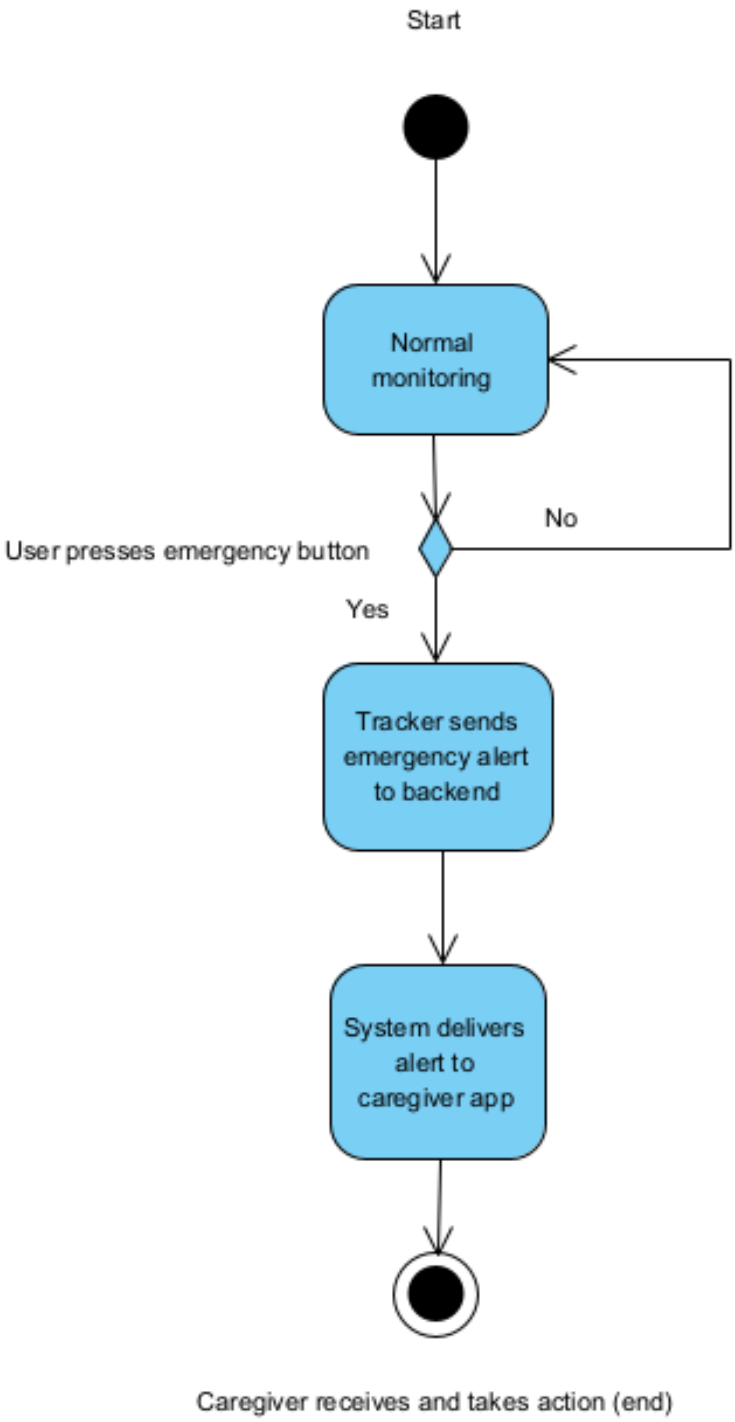

Figure 6.7: ucEmergencyAlertActivity: Activity Diagram for Emergency Alert (UC5)

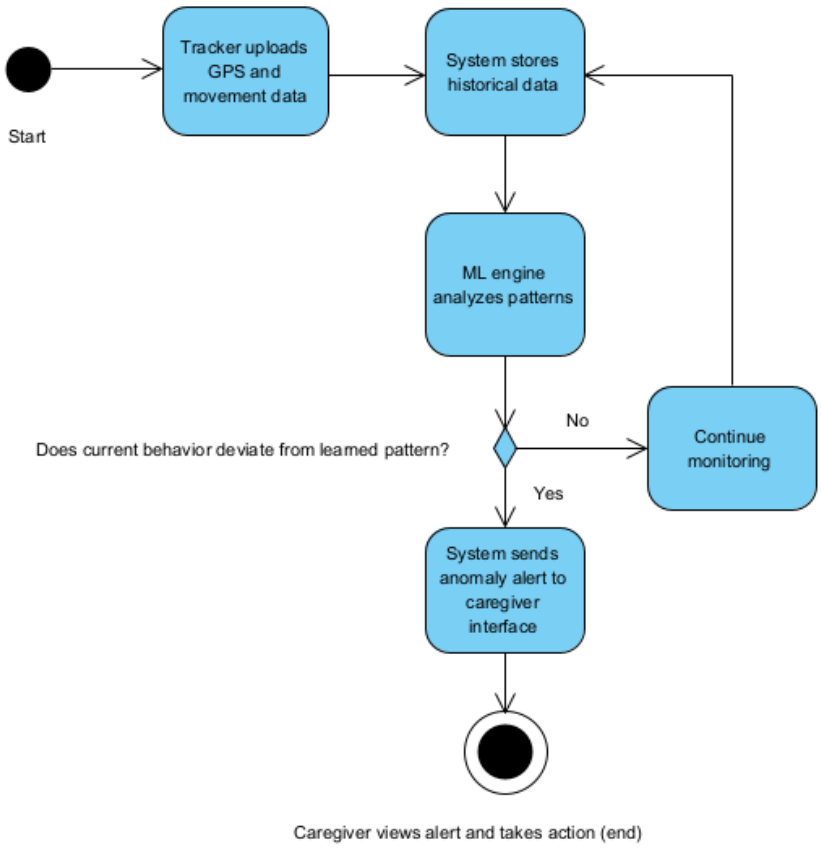

Figure 6.8: ucPatternDetectionActivity: Activity Diagram for Machine Learning Pattern Detection (UC6)

Chapter 7

User Interface Design

– Ryan Davis, Roy Tas, and Cory Vitanza

7.1 User Persona

Maria Davis

- **Age:** 42 years old
- **Family Status:** Full-time caregiver for her 80-year-old mother (early dementia)
- **Location:** Brooklyn, NY
- **Occupation:** Part-time dental assistant
- **Background:** Moved her mother in last year after wandering incidents
- **Core Challenge:** Balancing work, kids, and caregiving; high morning anxiety
- **Required Solution Features:**
  - Simple and dependable device
  - Immediate geo-fence notifications
  - Reduced need for continuous supervision

John Smith

- **Age:** 10 years old
- **Condition:** Diagnosed with autism
- **Location:** Paramus, NJ
- **Wandering Triggers:** Stressful or crowded environments (malls, airports)
- **Personality:** Smart, curious, active; enjoys technology
- **Device Preferences:**

  - Prefers small wearable devices
  - Dislikes bulky devices
- **Required Solution Features:**
  - Real-time location updates for parents
  - Quick distance + geo-fence alerts
  - Simple UI for school staff

Brian Cleary

- **Age:** 67 years old
- **Status:** Retired mechanic living alone
- **Location:** Weehawken, NJ
- **Health Concerns:** Dizziness; fall risk; forgets to charge devices
- **Routine:** Morning walks by the Hudson River
- **Device Preferences:**
  - Prefers extremely simple or automatic devices
  - Avoids smartphones
- **Required Solution Features:**
  - Long battery life
  - Automatic fall detection
  - Passive well-being updates for family

7.2 User Interface Design

Figma Paper Prototype: https://www.figma.com/make/jALgOM7hNgKhoa1sMRlw5L/User-Interface-for-Trackir
node-id=0-4

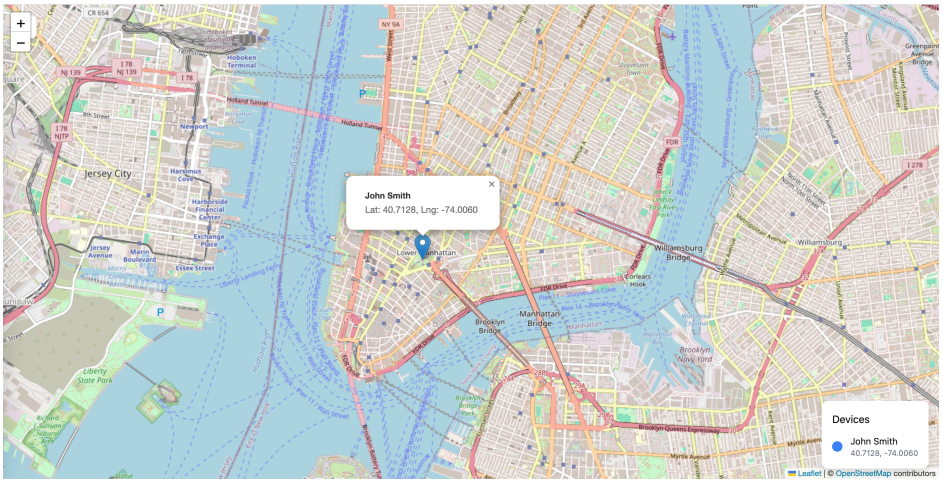

Figure 7.1: PaperPrototype: Figma Paper Prototype for FindIt UI.

Figure 7.2: PaperPrototype: Figma Paper Prototype of Side Menu.

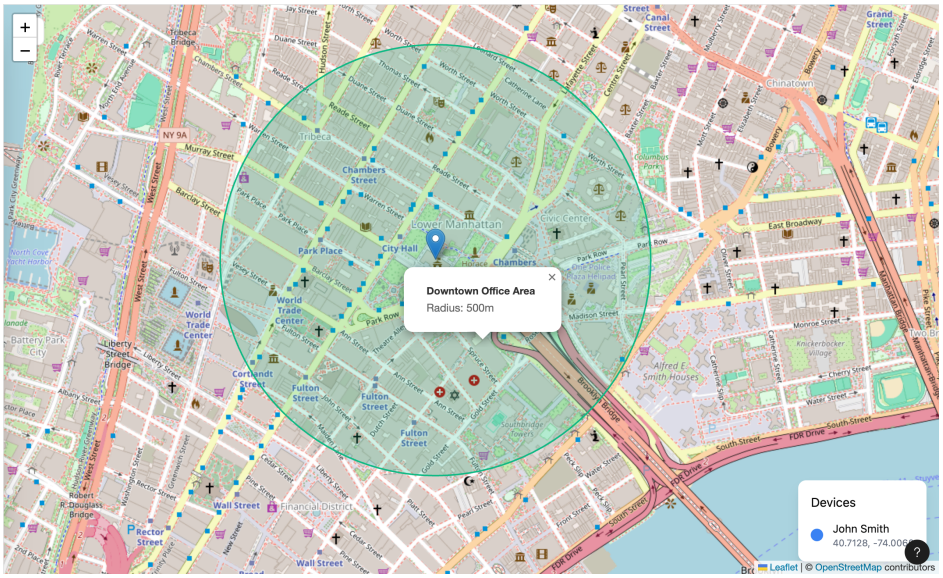

Figure 7.3: PaperPrototype: Figma Paper Prototype of Geofence Implementation.

Chapter 8

Logical View

– Ryan Davis, Roy Tas, and Cory Vitanza

Class diagrams present the structure of our Design, highlighting the firmware on our Wio Tracker 1110, and how that data is processed in our React software.

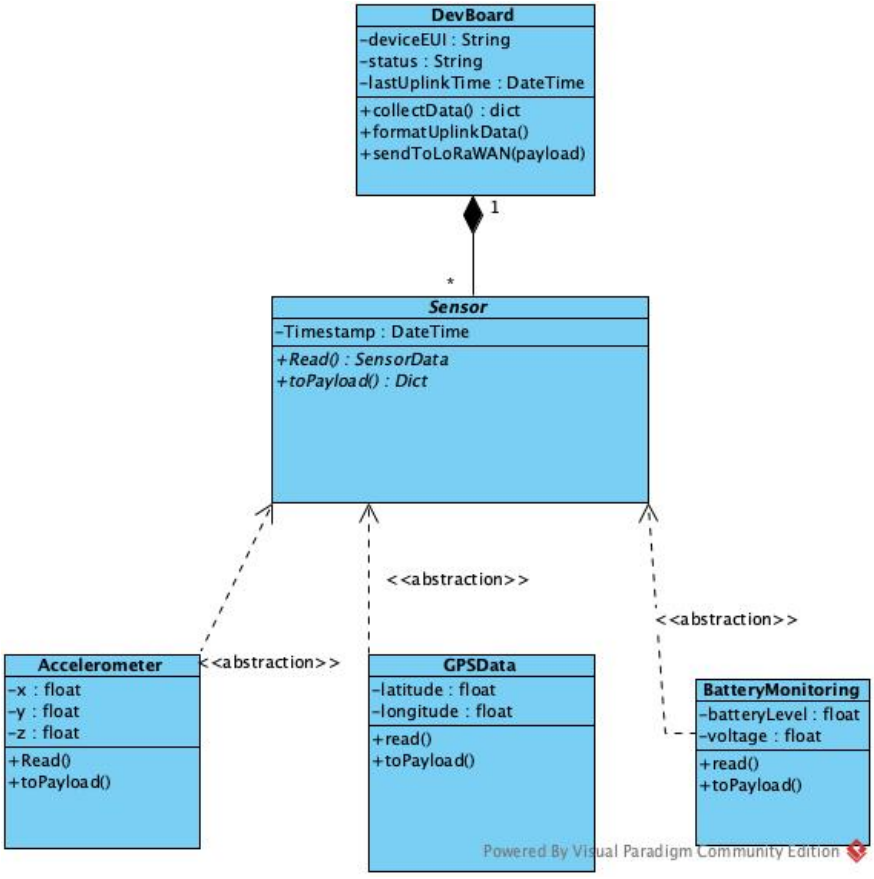

Figure 8.1: FirmwareClasses: Class diagram for the Wio Tracker 1110 Development Board's Firmware

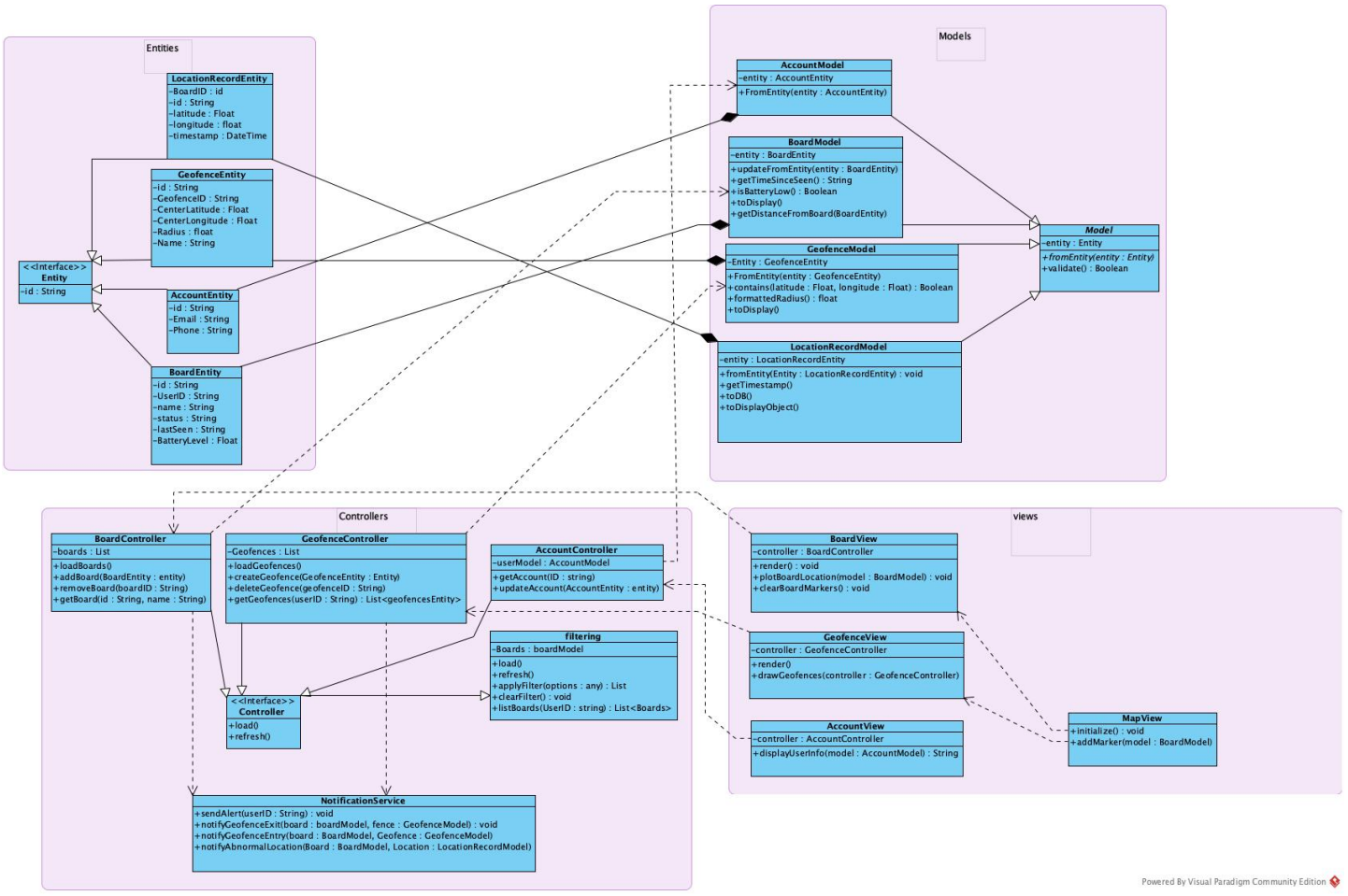

Figure 8.2: Model View Controller: Class Diagram for our React Interface Using the model view controller design pattern.

Class Diagram Description

- **Overview**
  - The class diagram is organized into four major packages: **Entities**, **Models**, **Controllers**, and **Views**.
  - This structure follows the Model–View–Controller (MVC) architectural pattern.
  - Each package contains classes with specific responsibilities and well-defined relationships such as inheritance, composition, realization, and dependencies.
- **Entities Package**
  - Contains raw data structures corresponding to the Supabase database tables.
  - Includes `BoardEntity`, `GeofenceEntity`, `AccountEntity`, and `LocationRecordEntity`.
  - All entities implement the `IEntity` interface, which standardizes the attribute `id` across all stored records.

  - Entities contain no domain logic and only represent database rows.

- **Models Package**
  - Models wrap the Entities through composition and extend the abstract `Model` base class.
  - Responsible for data cleaning, validation, and transformation (e.g., converting timestamps to `Date` objects).
  - Contain computed attributes required by the UI, such as geofence containment, distance calculations, battery warnings, and formatted display information.
  - Includes `BoardModel`, `GeofenceModel`, `AccountModel`, and `LocationRecordModel`.

- **Controllers Package**
  - All controllers implement the `Controller` interface.
  - Responsible for orchestrating system logic, including loading database data, updating models, applying filters, and managing CRUD operations.
  - Controllers detect events such as geofence breaches or location abnormalities, and trigger notifications via `NotificationService`.
  - Includes `BoardController`, `GeofenceController`, `FilteringController`, and `NotificationControll`

- **Views Package**
  - Views depend on their corresponding controller to retrieve and update models.
  - Views do not contain domain logic—their only responsibility is UI rendering.
  - Views interact with the `MapComponent` to display board locations, geofences, filters, and violations.
  - Includes `BoardView`, `GeofenceView`, `AccountView`, and the shared `MapComponent`.

- **Key Relationships**
  - **Inheritance:**
    * Model subclasses extend the abstract `Model`.
    * Controllers implement `IController`.
    * Views depend on `MapView`.
  - **Composition:**
    * Each Model class owns its corresponding Entity (e.g., `BoardModel` has a `BoardEntity`).
  - **Dependencies:**
    * Views depend on Controllers.
    * Controllers depend on Models and backend API calls.
    * Views depend on the shared `MapComponent` for UI rendering.
  - **Separation of Concerns:**
    * Views never access Entities or perform domain logic.
    * Models never render UI.

    * Controllers manage logic but never render UI directly.

- **Summary**
  - The diagram demonstrates a complete and maintainable MVC architecture.
  - Entities represent raw data, Models transform and prepare information for use, Controllers coordinate and implement CRUD, and Views render the user-facing interface.
  - This architecture supports geofence and board management, filtering operations, map visualization, and notification handling in the FindIt system.

Chapter 9

Development View

– Ryan Davis, Roy Tas, and Cory Vitanza

The package diagram for the FindIt system highlights the different modules in our system. The Wio Tracker 1110 has firmware, uplinking the data it collected into the database. The database provides data to our ML model highlighting abnormalities in location data, and to the React frontend, where the data is processed and presented on the user interface.

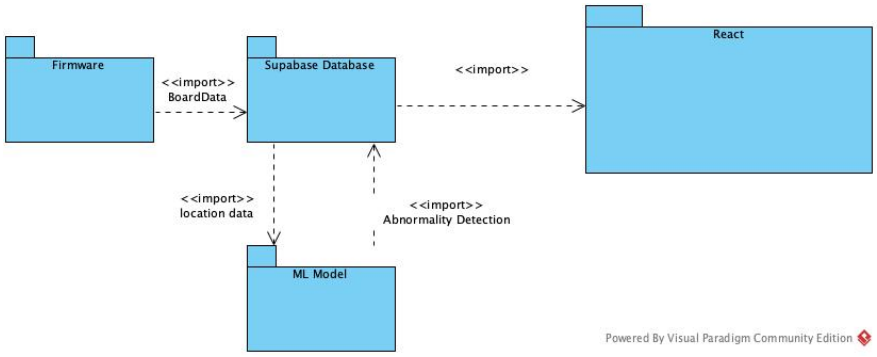

Figure 9.1: Package Diagram: Package diagram for FindIt deployment

Chapter 10

Process View

– Ryan Davis, Roy Tas, and Cory Vitanza

This sequence diagram shows the flows for our most prevalent use case, multi-resident monitoring. The activity starts with boards collecting gps data, and provides the end user with location data and notifications if a board leaves the geofence.

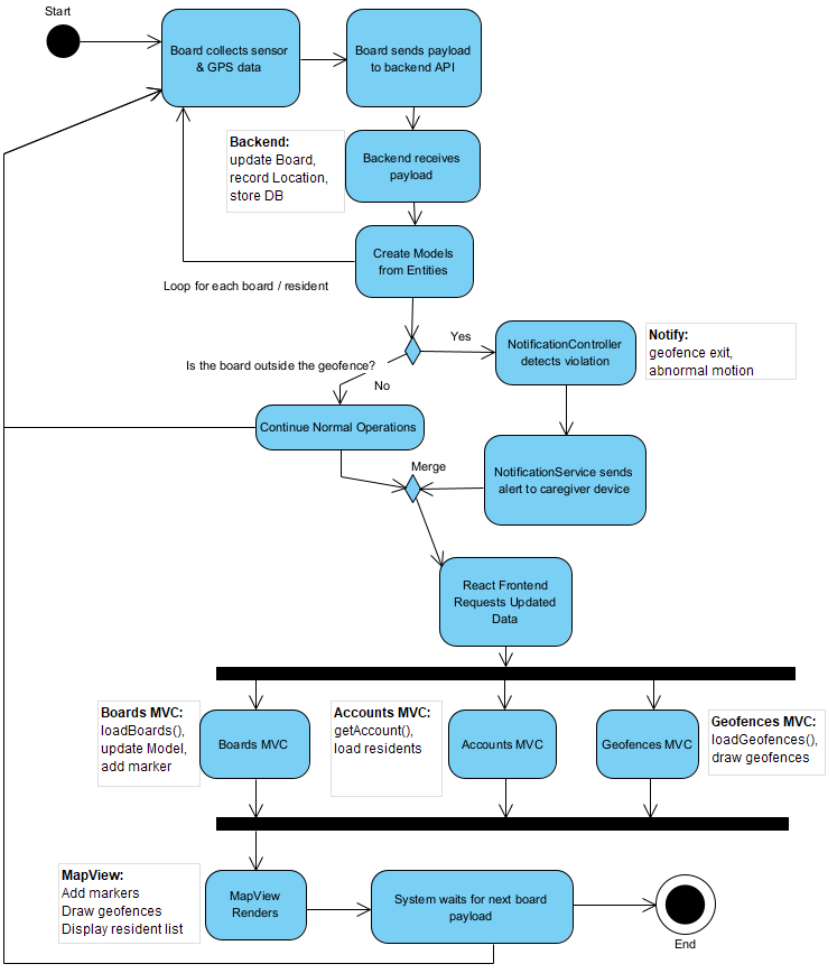

Figure 10.1: ucMultiResidentMonitoringActivity: Activity Diagram for Multi-Resident Monitoring (UC2)

Chapter 11

Physical View

– Ryan Davis, Roy Tas, and Cory Vitanza

FindIt component diagram depicting the interfaces for the different components of the system, including the development board, database, backend infrastructure, and react components.

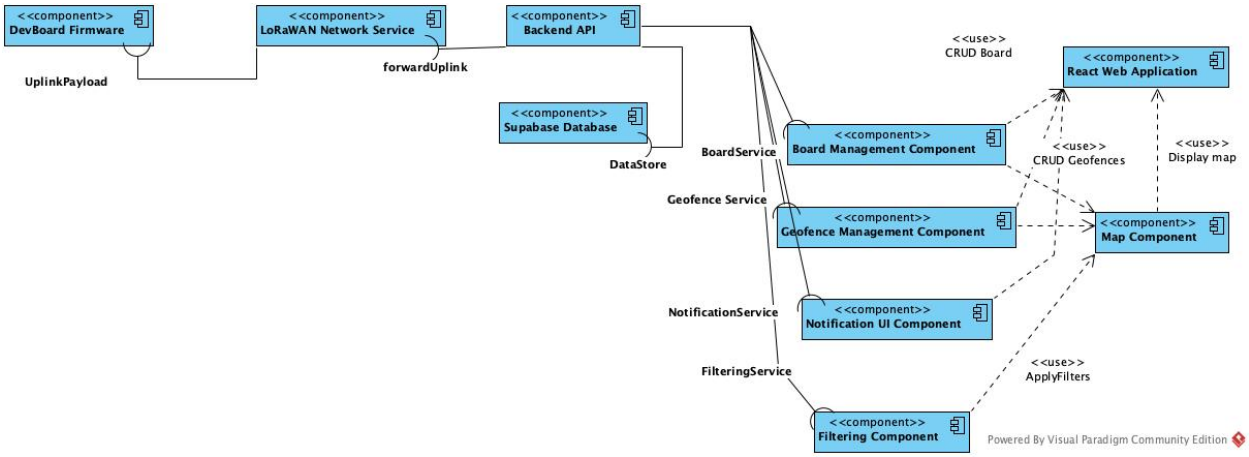

Figure 11.1: Component Diagram: Component diagram for FindIt deployment

Chapter 12

Glossary

Asset Tracking Monitoring the location of objects or people using GNSS (outdoor) and signal-based localization (indoor). 45

Battery Life Optimization Design focus on minimizing energy usage through duty cycling, GNSS assist, and efficient LoRaWAN classes to target ¿1 year per device. 45

CouldHave This defines the third highest priority requirement. The system could implement all of the tasks, requirements, or anything that is marked this way, but if resources are limited, it can be left out of the current and next version. Build in two versions from now. 14, 45

Device Downlink Network server transmits data to device. 13, 45

Device Uplink Device transmit data to network server. 13, 45

Emergency Communication User-initiated alerts (e.g., panic button or Morse-code sequence) transmitted via LoRaWAN for caregiver notification. 45

FindIt System The end-to-end asset tracking solution developed by the team, consisting of the FindIt device, LoRaWAN gateway, and cloud-based data management dashboard. 45

Firmware Embedded software on the tracker controlling sensing, power management, and LoRaWAN communication. 45

Gateway A device that receives LoRaWAN packets from trackers and forwards them to the network server via the internet, bridging local RF to cloud. 13, 45

Gateway Network Coverage Geographic area within which gateways can reliably receive tracker signals, determining communication reach and reliability. 45

GPS (Global Positioning System) Satellite-based navigation providing precise outdoor positioning. Used by the tracker when sky visibility is sufficient. 13, 45

Indoor Positioning Location estimation in GPS-denied spaces using RSSI triangulation, Wi-Fi fingerprinting, or Bluetooth beacons. 45

LoRa (Long Range) A wireless modulation technique that enables long-range, low-power communication between IoT devices using sub-GHz frequency bands. 45

LoRaWAN (Long Range Wide Area Network) A protocol built on top of LoRa that defines how end devices communicate with gateways and network servers for secure and scalable data transmission. 2, 13, 45

LR1110 A Semtech transceiver that supports LoRa and GNSS assistance, enabling hybrid indoor/outdoor tracking and low-power operation. 45

MustHave This defines the first highest priority requirement. All of the tasks, requirements, or anything that is marked this way are built in the current version. 14, 15, 16, 17, 45

One-Time Purchase Model Fixed device cost (e.g., $50) with optional subscription tiers for extended history, private hosting, or premium features. 45

RSSI (Received Signal Strength Indicator) Measured signal power used for distance/proximity inference and coarse indoor localization. 13, 45

ShouldHave This defines the second highest priority requirement. The system should implement all of the tasks, requirements, or anything that is marked this way, but if resources are limited, it can be left out of the current version. Build in next version. 14, 15, 16, 17, 45

Supabase The backend platform (PostgreSQL + REST/Realtime) used to store and query tracker data and power the FindIt API/dashboard. 13, 45

The Things Stack (TTS) An open-source LoRaWAN network server for device registration, session management, and secure routing of uplink/downlink messages. 13, 45

User Interface (UI) Mobile/web interface for viewing locations, receiving alerts, and managing settings. 45

Wio Tracker 1110 The development board used for prototyping the FindIt device. It integrates Semtech's LR1110 transceiver, GNSS assistance, and LoRa communication interface. 13, 45

WouldHave This defines the lowest priority requirement. The system would like to implement all of the tasks, requirements, or anything that is marked this way, but only if resources are available. It can be left out of all future versions. 45

Appendix A

Weekly Reports

- Ryan Davis, Cory Vitanza, and Roy Tas

A.1 Week Report 12 (12/05/2025)

A.1.1 Progress Made

- Received the Wio Tracker 1110 Development Board.
- Configured The Things Stack LoRaWAN network
- initialized Database Tables and schema

A.1.2 Next Week's Plan

- Configure the development board with Sensecap to collect device's keys for development.
- Debug the firmware of the device to optimize GPS collection and confirm LoRaWAN uplinks.
- Prepare User interface react application for the prototype.

A.2 Week Report 11 (11/14/2025)

A.2.1 Progress Made

- Created a Paper Prototype for the FindIt user interface.
- Developed user personas for the User Interface Design
- Began composing diagrams for our logical view (class diagram), development view (component diagram), process view (sequence diagram), and physical view (package diagram).

A.2.2 Next Week's Plan

- Upon the arrival of our development board we will begin developing the firmware for our Wio Tracker 1110.
- We will fully compile UML diagrams for the different views of the system.

A.3 Week Report 10 (11/7/2025)

A.3.1 Progress Made

- We initialized The Things Stack LoRaWAN network in preparation for our development board.
- We ordered our board again from a different vendor to decrease significant wait times.

A.3.2 Next Week's Plan

- We will begin working on our Preliminary design, highlighting User persona, UI design, Architecture Design, Use cases, Logical view, Development view, Process view, and Physical view.
- Regardless of the arrival of the development board, we will continue developing the front end of our application.
- We will collect board measurements in order to develop a case for our Wio Tracker 1110.

A.4 Week Report 9 (10/31/2025)

A.4.1 Progress Made

- Created activity diagrams that map to each of our 6 use cases, focusing on how the system can implement such use cases.
- Initialized The Things Stack LoRaWAN network server, preparing for the arrival of the Wio Tracker 1110 development board.

A.4.2 Next Week's Plan

- To continue our progress in development, we will continue refining our requirements and use cases. We will also create classes that correlate with our activity diagrams for the system.

A.5 Week Report 8 (10/24/2025)

A.5.1 Progress Made

- Initialized technology infrastructure, including SupaBase database, React frontend structure, Postman for API testing, and our continuous integration pipeline.
- Populated our Kanban project board, allowing for the designation of tasks regarding programming the development board, constructing the user interface, and creating the case for the development board.
- Added more activity diagrams to replicate the flows of our use cases

A.5.2 Next Week's Plan

- While awaiting for the arrival of the development board, we will construct the user interface, and begin testing the mapping of GPS coordinates.
- We plan to finalize our requirements, use cases, and user stories for the project concept development.
- We plan to create and present our Project Concept Development and Specification demo

A.6 Week Report 7 (10/17/2025)

A.6.1 Progress Made

- Created Use Case Diagram for the FindIt system.
- Created Activity Diagram based on our use cases.
- We completed the budget update form in order to order our Wio Tracker 1110 Dev board.

A.6.2 Next Week's Plan

- Depending on the Dev board's arrival, we plan on developing the board's logic with the Arduino IDE.
- We will begin developing our technology infrastructure, populating our Kanban board for development, and setting up essential tools like CircleCI and our SupaBase database.

A.7 Week Report 6 (10/10/2025)

A.7.1 Progress Made

- Updated Requirements chapter to perform stakeholder and domain analysis over our system.
- Added Use cases chapter outlining the different use cases for the FindIt system.
- Added User Stories illustrating the specific user base our system is intended for.
- Added Glossary defining essential terms to understand for the development of our product.
- Updated budget to account for new Wio Tracker Board.

A.7.2 Next week's Plan

- Develop system UML diagrams
- Meet with Dr. Damiano Zanotto
- Continue creating IT infrastructure (CI/CD, SupaBase).
- Await arrival of new Development board and debug.

A.8 Week Report 5 (10/02/2025)

A.8.1 Progress Made

This week, the team completed and presented our Demo, which covered our Project Mission and Development Plan. We received constructive feedback from professors and peers, which will help us refine our development strategy. In addition, the team set up our GitHub repository and established a project Kanban board within GitHub Projects to track tasks and streamline collaboration in an Agile workflow.

A.8.2 Next Week's Plan

For next week, the team plans to order the new development board and begin work on multiple components of our system. This includes initiating development of the frontend and user interface, setting up the Supabase database for data storage, and integrating CircleCI for continuous integration with our GitHub repository. We will also begin configuring our DevOps infrastructure to support efficient development and deployment workflows.

A.9 Week Report 4 (09/25/2025)

A.9.1 Progress made

This week, our team has decided that we have to purchase a new board. This new board will remove Amazon Sidewalk capabilities, allowing us to narrow our focus on the LoRaWAN capabilities of the development board. Unfortunately, our current board's bootloader is configured to the wrong board-preset, thus preventing us from accessing certain areas of our current board. Our current board thinks its a Seeed XIAO board, but in reality, it is a WIO Tracker 1110 development board. This change will further allow us to build our tracking unit.

A.9.2 Next Week's Plan

Next week, the team plans to complete our 1st presentation, highlighting our project mission and development plan for this technology stack. We also plan to acquire the new board, start creating our GitHub, along with the agile board and task assignments for our members. We will begin to start developing the board to receive gps data, and establish our pipeline for data transmission.

A.9.3 Risk Mitigation

As we implement our tracking algorithm into the development board, a risk that we would like to minimize is the accuracy of the tracking data. We aim to implement indoor and outdoor tracking data, allowing for improved precision in location data.

A.10 Week Report 3 (09/18/2025)

A.10.1 Progress made

This week, the team was able to compose our Team Declaration, which includes an outline of our proposed project, our mission, key drivers for our design, and key constraints to keep in mind throughout our development.

A.10.2 Next Week's Plan

Next week, the team plans to debug the current Seeed Studio LoRaWAN Arduino device, through configuring the bootloader to ensure complete functionality of the board, including the on-board buttons, GPS module, LoRA module, and the serial connection. The team will also begin to compose our project mission and development plan for the upcoming presentation.

A.10.3 Risk Mitigation

The risk that the team is currently facing coincides with the debugging of the LoRaWAN device. Currently, the device is flashed with print statements to the serial monitor, but is constrained to this operation without access to other board features, such as button usage and the ability to flash the device again. To minimize this risk, we are looking into alternative devices, such as a Raspberry Pi Nano, but will also continue to attempt to debug the current Seeed Studio device.

A.11 Week Report 2 (09/12/2025)

A.11.1 Progress Made

This past week, the team created a deployment diagram that illustrates the system's interfaces and necessary developments. The team also met with Professor Damiano Zanotto to further expand and outline the scope of our asset tracking project. We are moving in the direction of developing this low-bandwidth and inexpensive tracker for people with autism or dementia patients who may wander, allowing caregivers and family members to locate the tracker. We are also looking to host the data locally, directly from the machine using The Things Stack, which is a LoRaWAN network server that allows API retrieval of data. We would also like to look into implementing a machine learning model which can process GPS data for precise indoor and outdoor geo-location tracking.

A.11.2 Next Week's Plan

For next week, the team has split the planning of our development into different components. Ryan's research will involve looking into open source tracking projects that can be tailored towards our model. Roy's research will look into the scalability of our model, and the extent to which our system will be built out, and Cory's research will explore how our model can avoid using services like AWS to minimize the transactions of our data. We will also continue to debug the LoRaWAN machine that we are working with, and will also explore our other options with such devices.

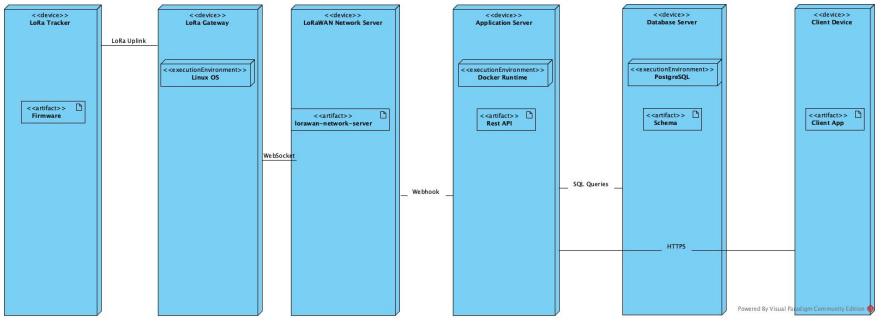

Figure A.1: System Diagram

A.12 Week Report 1 (09/05/2025)

A.12.1 Progress made

Our group which currently consists of 2 members, Ryan Davis and Cory Vitanza, are currently exploring our options regarding our project. We have reached out to classmates and professors and plan on making our decision within the next week.

A.12.2 Next Week's Plan

Following my Summer Research with Doctor Damiano Zanotto, I plan on requesting my Summer research project to be extended throughout the course of my senior design project. This project includes asset tracking using embedded minicomputers communicating over a LoRaWAN connection in order to process gps data.

A.12.3 Backup Plan

If this plan does not work in our favor, we plan to reverse engineer and develop a ”magic mirror”, which acts as a dashboard, containing different modules showing a handful of information. This is driven by a raspberry pi board over an internet connection, further displayed on a monitor and a 1-way mirror.

Appendix B

Development Notes

- Ryan Davis, Cory Vitanza, and Roy Tas

B.1 Wio Tracker 1110 configs

- Boards manager URL for ArduinoIDE: https://files.seeedstudio.com/arduino/package_seeeduino_boards_index.json
- Boards file: Seeed nRF52 Boards
- Seeed Wio Tracker 1110

B.2 Connecting to TTN

- https://wiki.seeedstudio.com/connect_wio_tracker_to_TTN/
- JoinEUI=AppEUI
- sensecraft app for enrollment.

Bibliography

[1] N. Hashim, F. Idris, T. N. A. Tuan Ab Aziz, S. H. Johari, R. Mohd Nor, and N. Ab Wahab, "Location tracking using LoRa," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 11, no. 4, pp. 3123–3128, 2021. [Online]. Available: https://doi.org/10.11591/ijece.v11i4.pp3123-3128

[2] L. Quaglietta, B. H. Martins, A. de Jongh, A. Mira, and L. Boitani, "A low-cost GPS GSM/GPRS telemetry system: Performance in stationary field tests and preliminary data on wild otters (*lutra lutra*)," *PLoS ONE*, vol. 7, no. 1, p. e29235, 2012. [Online]. Available: https://doi.org/10.1371/journal.pone.0029235

[3] Ç. Civelek, "Development of an IoT-based (LoRaWAN) tractor tracking system," *Journal of Agricultural Sciences (Tarım Bilimleri Dergisi)*, vol. 28, no. 3, pp. 438–448, 2022. [Online]. Available: https://doi.org/10.15832/ankutbd.769200

[4] Muladi, H. Wijaya, S. D. Prasetyo, S. A. Hamzah, and A. K. Mahamad, "LoRa mesh-based IoT GPS tracking system for mountain climbers," *International Journal of Safety and Security Engineering*, vol. 14, no. 6, pp. 1751–1761, 2024. [Online]. Available: https://doi.org/10.18280/ijsse.140610

[5] M. Arebey, M. A. Hannan, H. Basri, R. A. Begum, and H. Abdullah, "Integrated technologies for solid waste bin monitoring system," *Environmental Monitoring and Assessment*, vol. 177, no. 1-4, pp. 399–408, 2011. [Online]. Available: https://doi.org/10.1007/s10661-010-1642-x

Index